Práctica 1

Manuel Suano Gallardo

Tarea CoNLL-2002

La tarea CoNLL-2002 se refiere al reconocimiento de entidades con independencia del idioma. En esta tarea se definieron cuatro tipos de entidades: personas (PER), localizaciones (LOC), organizaciones (ORG) y otro tipo de entidades (O). Además de la etiqueta del tipo de entidad, para aquellos tokens que pertenezcan a una diferente de O también se indicará si es el primer token de la entidad, añadiendo B a la etiqueta, o si no lo es, añadiendo I a la etiqueta.

Se utilizaron dos corpus: uno en español y otro en holandés.

- El corpus español era una colección de noticias de mayo del 2000 de la Agencia EFE, con un tamaño de 273037 líneas para el corpus de entrenamiento, 54837 líneas para el de validación y 53049 líneas para el de prueba. La anotación la llevaron a cabo en la Universidad de Barcelona y en la Universidad Politécnica de Cataluña.
- El corpus holandés se trata de cuatro ediciones del 2000 del periódico belga “De Morgen”. Los conjuntos de entrenamiento, validación y prueba constan de 218737, 40656 y 74189 líneas respectivamente.

Para la evaluación de los modelos se usó las métricas precisión, recall y la $F_{\beta=1}$. La precisión es el porcentaje de entidades que se etiquetan correctamente y el recall es el porcentaje de entidades presentes en el corpus y que son encontradas por el sistema.

Se probaron 12 sistemas diferentes y se comprobó con un sistema de referencia. Los diferentes sistemas incluyeron modelos como: máquinas de vectores de soporte, métodos basados en transformaciones que generaban reglas, árboles de decisión, concatenación de clasificadores que se retroalimentan, modelos de Markov, modelos de máxima entropía, entre otros.

Lo que se pudo observar es que el modelo que mejor realizaba el etiquetado de las entidades era la combinación de árboles de decisión, con una profundidad fijada, usados en conjunto con clasificadores AdaBoost, desarrollado por Xavier Carreras, Lluís Màrques y Lluís Padró. De forma general, la utilización de clasificadores en cascada mejoraba el etiquetado.

Por otro lado, se observó que los modelos que peores resultados ofrecían eran las máquinas de vectores de soporte para el caso del español y el método basado en transformaciones para el caso del holandés.

Desarrollo de la práctica

El objetivo de la práctica es evaluar el desempeño del modelo preentrenado que se dispone en Spacy.

Primero se comienza cargando el corpus al completo (archivo `esp.testb`), separando los tokens y las etiquetas en dos arrays para que sea más sencillo el manejo.

```
tokens = []
etiquetas = []
with open('./esp.testb', 'r') as corpus:
    for linea in corpus:
        linea = linea.strip()

        if not linea:
            continue

        division_linea = linea.split(' ', maxsplit=1)

        tokens.append(division_linea[0])
        etiquetas.append(division_linea[1])
```

En el momento de la lectura del archivo, aquellas líneas que sean vacías se saltan. Las líneas no vacías se dividen por el espacio que separa el token de la etiqueta.

El total de tokens leídos son 51.533, que resulta de eliminar las líneas en blanco (1.516) del total de líneas del archivo (53.049).

La extracción del texto plano se puede realizar a partir del array tokens:

```
texto_plano_espaciado = " ".join(tokens)
```

Este texto ya puede ser usado en el etiquetador. Para ello:

```
nlp = spacy.load("es_core_news_lg")
doc = nlp(texto_plano_espaciado)
```

A partir de la variable doc obtenemos el etiquetado de los tokens que ha detectado el etiquetador.

```
etiquetas_pred = []
tokens_etiquetados = []

for token in doc:
     tokens_etiquetados.append(token.text)
```

```
        etiquetas_pred.append(f"{token.ent_iob_}{f'-
{token.ent_type_}' if token.ent_type_ else ''}")
```

A partir de las etiquetas predichas y las originales, usando la función `evaluate` del módulo `conlleval` proporcionado, podemos obtener las métricas precisión, cobertura y F1:

```
evaluate(etiquetas, etiquetas_pred)
```

El resultado es: 2.35% de precisión, 2.64% de cobertura y 2.48 de F1. Como se puede ver, son resultados muy malos. Procedemos a ver que puede estar ocurriendo.

Comenzamos comprobando si el etiquetador ha generado el mismo número de tokens que teníamos en el archivo:

```
print(len(etiquetas), len(etiquetas_pred))
```

Como podemos ver, el número de tokens que ha detecta el etiquetador es superior al que se tenía en el archivo: el etiquetador detecta 228 tokens más. Este parece ser el principal motivo de esos valores tan bajos en las métricas, pues al generar más tokens existe un desajuste entre la etiqueta original y la que le correspondería predicha. Si buscamos donde ocurre por primera vez este desajuste entre el token definido en el corpus y el token detectado por el etiquetador vemos que, por un lado, en el corpus está el token “20.” mientras que el etiquetador realiza la separación del punto del 20.

```
for ind in range(0,len(etiquetas_pred)):
      if tokens_etiquetados[ind] != tokens[ind]:
            print(tokens[ind],
                  tokens_etiquetados[ind],
                  tokens_etiquetados[ind+1])
      break
```

Si buscamos este caso en el corpus usando la expresión regular “`\d+\.`” vemos que ocurre 18 veces. Suponiendo que el modelo no podemos reentrenarlo, podemos realizar un nuevo etiquetado de forma automática, sin alterar el corpus, en el momento que estemos leyéndolo. En este caso, podemos transformar las instancias que cumplan dicha expresión regular simplemente separando el punto y asignándole la misma etiqueta. En algunos casos, este punto junto con al número realmente está haciendo referencia a la conjunción.

Esta búsqueda de tokens discordantes entre el corpus y el etiquetador se puede repetir para conocer los casos e ir etiquetando de nuevo los diferentes casos. Otros casos que realiza el etiquetador a diferencia de cómo está en el corpus son:

- Dada una expresión "\d+,\d+" correspondiente a un número decimal en español, se detecta como números independientes. Para tratar esto se harán dos casos: si se detecta un número superior al millar con un punto (expresión "\d+\.\d+") se eliminará el punto, y si se detecta un número con coma se sustituirá por un punto, para adaptarlo al formato inglés.
- Dada una expresión "[a-zA-Z]+[,\.:]$", se divide entre la palabra y la coma. Para el tratamiento de este caso se tendrá en cuenta la categoría del token en el corpus: si tiene categoría O o I-xxx, se seguirá el mismo proceso que el descrito para "\d+\.". Para la categoría B-xxx hay que asignar B-xxx al primer término
- Dada una expresión "[a-zA-Z]/.*" se separa la barra del texto. Hay casos en los que aparece más de una vez, otros una sola vez en medio, otros al final.
    - Si la expresión es "[a-zA-Z]/", se aplica el mismo caso que el de la expresión "\d+\.".
    - Si la expresión es "[a-zA-Z]+\/[a-zA-Z]+$", se separa por la barra y se añaden las dos expresiones textuales y la barra. Habrá que tener en cuenta si la etiqueta es O o si es I-xxx, que en ese caso solo hay que repetir la etiqueta. Si es B-xxx se indicará a B-xxx la primera expresión y las otras dos a I-xxx.

Estos son solo algunos casos realizados. Solo realizando estos nuevos etiquetados se consigue aumentar las métricas a un 20.6% para la precisión, 23.27% para la cobertura y 21.86 para la F1, además de reducir la diferencia entre la tokenización del modelo y la del corpus a solo 86 de diferencia.

Este nuevo etiquetado se hace dinámicamente con bloques `if` dentro del bucle `for lineas in corpus`:

```
# Si {número}[. o ,]{número}
 if re.search(r'\d+[,\.]\d+', token):
     token = token.replace('.', '')
     token = token.replace(',', '.')
     tokens.append(token)
     etiquetas.append(etiqueta)
# Si ({número}[.])
#   o ({texto}[, o . o :] y etiqueta es O o I-xxx)
#   o ({texto}/)
elif (
     re.search(r'\d+\.', token) or (
          re.search(r'[a-zA-Z]+[,\.:]$', token) and
          (etiqueta == 'O' or etiqueta.startswith('I'))
     ) or
```

```
            re.search(r'[a-zA-Z]/$', token)
        ):
            tokens.append(token[:-1])
            tokens.append(token[-1])
            etiquetas.append(etiqueta)
            etiquetas.append(etiqueta)

        # Si {texto}[, o . o :] y etiqueta es B-xxx (no hace falta
        comprobación)
        elif re.search(r'[a-zA-Z]+[,\.:]$', token):
            tokens.append(token[:-1])
            tokens.append(token[-1])
            categoria = etiqueta.split('-')[1]
            etiquetas.append(f"B-{categoria}")
            etiquetas.append(f"I-{categoria}")

        # Si {texto}/{texto}
        elif re.search(r'[a-zA-Z]+\/[a-zA-Z]+$', token):
            div_token = token.split('/')
            tokens.append(div_token[0])
            tokens.append('/')
            tokens.append(div_token[1])
            if etiqueta == 'O' or etiqueta.startswith('I'):
                etiquetas = etiquetas + [etiqueta] * 3
            else:
                categoria = etiqueta.split('-')[1]
                etiquetas.append(f"B-{categoria}")
                etiquetas = etiquetas + [f"I-{categoria}"] * 2

        # caso normal
        else:
            tokens.append(token)
            etiquetas.append(etiqueta)
```

Además de estos casos, también existen errores de etiquetado por parte el modelo, como es el caso de las dos primeras palabras del corpus “La Coruña”. En este caso, el modelo etiqueta “La” como O y “Coruña” como B-LOC, cuando las etiquetas correspondientes sería B-LOC e I-LOC respectivamente. Esto puede deberse a que el modelo no conoce que la ciudad es “La Coruña” y no solamente “Coruña”, por lo que interpreta “La” como un simple determinante.

Otro caso es el de “los clientes”: en el corpus están etiquetados con O pero, sin embargo el modelo los reconoce como miscelánea, indicando B-MISC e I-MISC

respectivamente, interpretando que se tratan de sujetos con suficiente relevancia como para que sean una entidad nombrada.

Más casos de etiquetados erróneos relevantes es para el texto “El debut de la selección paraguaya”, que en el corpus está etiquetado con O cada palabra y el modelo lo detecta como un evento con cierta relevancia y lo etiqueta como entidad nombrada, usando MISC.